FIPS 206 Status Update

Ray Perlner
Computer Security Division
NIST

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY
U.S. DEPARTMENT OF COMMERCE

Where FIPS 206 (FN-DSA) stands

After working on FIPS 206 for a couple years in consultation with the FALCON team (Thank you!):

We expect to release an Initial Public Draft soon

- It's basically written, awaiting approval

This talk will preview what to expect from FIPS 206 IPD

- FN-DSA/FALCON review
- Special challenges relating to floating point
- Similarities/Differences to other FIPS
- Planned changes from FALCON as submitted

FIPS 206 and FN-DSA

FIPS 206 specifies FN-DSA based on the selected NIST PQC submission FALCON

- A lattice-based signature scheme
- In the Hash-Then-Sign paradigm
  - Signature is a lattice point close to target point $\boldsymbol{t}$ derived from a randomized message hash
- Uses NTRU Lattice over the NTT-friendly ring $\mathbb{Z}_q[X]/\langle X^n + 1\rangle$, where $n = 2^\ell$
- Uses FFT, LDL tree (FALCON Tree) to approximate discrete Gaussian sampling

FN-DSA has very small signatures and public keys but is difficult to implement

- Significantly smaller than ML-DSA (FIPS 204)
- Operations in KeyGen and Signing need floating-point arithmetic
- This can lead to challenges for validation and side-channel protection

FALCON details

KeyGen

- Private key will be a lattice basis (row notation) $\mathbf{B} = \begin{pmatrix} g & -f \\ G & -F \end{pmatrix}$
  - Sample short (Gaussian) $f, g \in \mathbb{Z}[X]/\langle X^n + 1 \rangle$
  - Compute F, G that satisfy NTRU equation: $fG - gF = q$
  - Check Gram-Schmidt(GS) Norm less than $1.17\sqrt{q}$
  - Use *reduce* to find equivalent basis where F, G are small enough to byte-encode
- If something fails, try again
- If everything succeeds, $h = f^{-1}g$ is public key and $\mathbf{B}$ is private key

Signing

- Expand private key into $\widetilde{\mathbf{B}} = \mathrm{FFT}(\mathbf{B})$ and LDL tree T
  - LDL tree is computed from $\widetilde{\mathbf{B}}$
  - Correct signature distribution requires numerical value at leaves of T between $\sigma_{\min}$ and $\sigma_{\max}$
  - GS-norm check should guarantee this (but note floating-point isn't exact!)

Our approach to GS-norm, $\sigma_{\min}$, and $\sigma_{\max}$ (provisional)

Keygen:

- Modify GS-norm check: New max value: $0.9999 \cdot 1.17\sqrt{q}$

LDL-tree (officially, part of signing):

- Explicitly check LDL leaves between $\sigma_{\min}$ and $\sigma_{\max}$
- If not, signature always returns $\perp$

Our philosophy on floating point/validation (provisional)

Verification

- No floating-point arithmetic
- Implementations expected to exactly match KATs

Table 1. Allowable representations in FN-DSA

	Key Generation	$\widetilde{\mathbf{B}}$ and LDL Tree	Signing	Verification
Floating point	Allowed	Allowed	Allowed	No - N/A
Emulated floating point	Allowed	Allowed	Allowed	No - N/A
Fixed point	Allowed	No	No	No - N/A

Signing

- Uses native or emulated floating-point arithmetic (IEEE-754 binary64 – *relevant excerpts included with permission in FIPS*)
- Correct distribution of signatures is essential to avoid private-key leakage
  - Therefore, implementations are expected to exactly match KATs
  - In support, we specify details (order of operations, no fused multiply-add…)
- FFT private basis and LDL tree can be cached (but these are part of signing for validation purposes)

Key Generation

- Exact match to KATs not needed for validation
  - Check GS norm, NTRU equation, infinity norm, correct generation of f, g from seed
- This is OK, since small deviation in distribution of private keys has limited effect on security
- Allow native/emulated floating point **or fixed point**
- Allow smaller loop limits (e.g. for avoiding timing side channels)

Our default encodings for Public/Private Keys
(provisional)

Public Key

- Use NTT form: $\hat{h} = (NTT(f))^{-1} NTT(g)$
- Saves a little bit of computation, and no real downside

Private Key

- Use f, g, F (also PK hash tr)
- G can be recomputed as $Ff^{-1}g \bmod^{\pm} q$
- Do not use seed as private key
  - Variations in KeyGen implementation can result in different keypairs from same seed
- OK to cache LDL tree and $\widetilde{\mathbf{B}}$, but discouraged to export these

Comparison to FIPS 204, 205
(provisional)

Similarities

- Separate pure/prehash versions
- Separate internal/external keygen, sign, verify algorithms
- Uses little-endian encodings for numerical values
- Uses SHAKE for pseudorandom sampling
- Includes BUFF transform

Differences

- Only allows randomized signing
  - Deterministic signing could be dangerous: mistakes in floating-point implementation could cause different signatures for same hash
- Forbids export of seeds
  - Keygen does not guarantee seeds will always expand the same way
- Discourages export of FFT basis and LDL tree
  - These would encode floating-point values, and we don’t specify a standard byte order for this

Other planned changes
(provisional)

- Signing and Verification restrict infinity norm of signatures in addition to L2 norm
  - Suggested by Yang Yu as described by FALCON update last workshop
- Base sampler samples 79 bits of randomness per coefficient instead of 72
  - Uses extra 7 bits from byte used to sample sign bit. No additional XOF calls
- While neither of the above changes is believed to be necessary for security, both have minimal cost and can't hurt security
- Also, made pseudorandom seeds uniformly 40 bytes (except keygen uses 32 bytes)

Possible areas for feedback

- Any objections to the changes discussed so far?
- Should we add support for key/message-recovery mode?
- Should we add support for a fixed-point version of signing?
- Regarding the recent “Do Not Disturb a Sleeping Falcon” paper (Eurocrypt, 2025, https://eprint.iacr.org/2024/1709.pdf)
  - We think our validation approach for signing, and our requirement for randomized signatures prevents these attacks
  - But should we also consider additional changes suggested by that paper?

Thank You!

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY
U.S. DEPARTMENT OF COMMERCE

Bonus Slide: References for Constant Time Implementation

Thomas Pornin, *New Efficient, Constant-Time Implementations of Falcon,* *https://eprint.iacr.org/2019/893.pdf*

J. Howe T. Prest, T. Ricosset, M. Rossi, *Isochronous Gaussian Sampling: From Inception to Implementation With Applications to the Falcon Signature Scheme, PQCrypto2020,* *https://eprint.iacr.org/2019/1411.pdf*

Thomas Pornin, *Falcon on ARM Cortex-M4: an Update,* *https://eprint.iacr.org/2025/123*